Week 3 Plan:

Week 3 Roadmap (small missions, clean architecture)

Mission A — Repo & code organization (no “big code” yet, just structure)

Goal: you don’t want “one big script”. You want a pipeline you can extend.
Deliverables:

- `src/` structure (example):

Code

```
src/
  config.py
  io/
    paths.py
    json_utils.py
  video/
    ffmpeg_extract.py
    shots.py
  lvvm/
    client.py
    prompts.py
    parse.py
  algorithm/
    normalize.py
    complexity.py
    phase.py
  pipeline/
    run_video.py
```

Rules

- One module = one responsibility.
- Pure functions where possible.
  Every output is written under `outputs/<video_id>/...`

Mission B — Standardize “Signature” + “Interpretation” schemas (design)

Goal: lock your JSON contracts so everything downstream is stable.

1) `VideoSignature.json`

Minimal fields:

```json
{
  "video_id": "...",
  "duration_seconds": 8.6,
  "shot_count": 12,
  "object_tracks_count": 34,
  "interaction_tracks_count": 7,
  "human_presence_ratio": 0.42,
  "object_entropy": 2.7
}
```

This is “Week 2 style” numeric metadata (mostly from GCP / pre-processing).

2) `VideoInterpretation.json`

```json
{
  "video_id": "...",
  "features_raw": {...},
  "features_norm": {...},
  "scene_complexity_score": 0.63,
  "scene_complexity_breakdown": {
    "shot_frequency": 0.22,
    "object_entropy": 0.25,
    "interaction_density": 0.11,
    "human_presence_ratio": 0.05
  },
  "narrative_phase": "Dynamic",
  "phase_reasons": ["sf > 0.65 and dens <= 0.65"],
  "lvvm_summary": {
    "model": "glm-4.6v-flash",
    "prompt_version": "v1",
    "text": "..."
  }
}
```

This is “Week 3 algorithm output” (score + phase + reasons + breakdown)

Deliverable: a short README/design-doc note that defines:

- the 4 features
- the v1 normalization ranges
- the phase rules
- the outputs above

Mission C — Harden the LVLM call into a reusable component (code)

Right now it's a demo script. Convert it into a library-style client.

Small steps:

1. lvvm/client.py: call_model(images_b64, prompt) -> text
2. lvvm/prompts.py: keep prompts versioned (PROMPT_V1 = ...)
3. lvvm/parse.py: optional (later) structured extraction; for now store raw text.

Non-negotiable hygiene:

- API key via env: ZAI_API_KEY
- consistent error handling (your retry logic is already good)
- save lvvm_summary.text into JSON for traceability

Mission D — Keyframe policy (code + light experimentation)

Goal: sampling strategy must be consistent, defendable, and cheap.

v1 policy (recommended)

- fps = 1
- max_frames = min(10, ceil(duration))
- evenly spaced timestamps (what you already do)

Deliverables:

- video/ffmpeg_extract.py exports extract_frames_b64(video_path, fps, max_frames)
- unit-ish sanity: print how many frames extracted + timestamps

Mission E — Compute the missing feature: shot_frequency (code)

You already planned it:

- shot_frequency = shot_count / duration_seconds

Deliverable:

- store inside features_raw (either in Signature or computed in algorithm layer)

Mission F — Normalize features (code)

Implement your v1 normalization exactly once.

Deliverable

- algorithm/normalize.py (V)
    - clamp01(x)
    - minmax_norm(x, minv, maxv)
    - normalize_features(raw_features) -> normalized_features

Use your v1 ranges:

- **shot_frequency:** 0–2 cuts/sec (sf) - How often the camera cuts (changes shot) in the video.
- **object_entropy:** 0–4 (ent) - How diverse(many types of objects) the objects are in the scene.
- **interaction_density:** 0–10 tracks/sec (dens) - How much "activity" is happening in the scene.
- **human_presence_ratio:** already 0–1

Also define:

- **interaction_density** = interaction_tracks_count / duration_seconds (if you aren't already computing it)

Mission G — SceneComplexityScore (code)

Implement v1 exactly as you wrote, plus breakdown contributions.

Deliverable:

- algorithm/complexity.py (V)
- compute_complexity(norm_features) -> (score, breakdown_dict)

Mission H — NarrativePhaseDetector (code)

Implement your rules, plus "reasons".

Deliverable:

- algorithm/phase.py (V)
- classify_phase(norm_features) -> (phase, reasons[])

Rules (v1):

- Dense if (ent > 0.65 and dens > 0.65) - Scene is crowded + complex.
- Dynamic if (sf > 0.65 and dens <= 0.65) - Scene is fast-paced but not crowded.
- Static if (sf < 0.35 and dens < 0.35 and ent < 0.35) - Scene is very calm / minimal.
  else Calm - everything else - Balanced / normal scene.

Mission I — Orchestrate pipeline: one command → outputs (code)

Goal: python -m src.pipeline.run_video <path> produces:

- outputs/<video_id>/VideoSignature.json
- outputs/<video_id>/VideoInterpretation.json

Deliverable:

- pipeline/run_video.py:

1. load/probe duration
2. get / load shot_count (from your existing method; placeholder if not ready)
3. extract frames
4. LVLM summary text
5. build raw features
6. Normalize
7. complexity + phase
8. write both JSONs

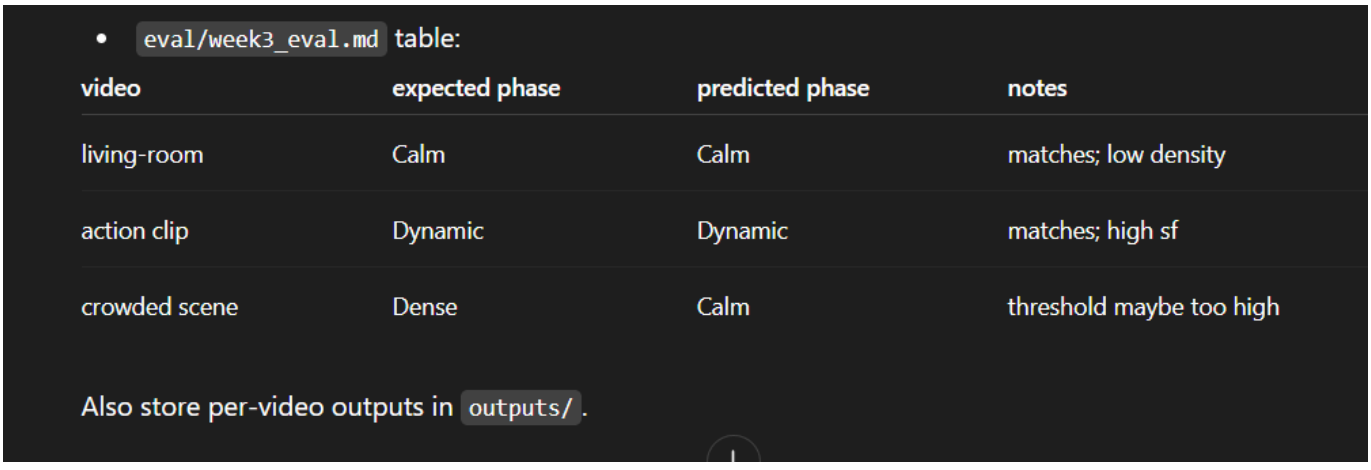
Keep it boringly deterministic.

Mission J — Minimal evaluation harness (testing + markdown)

Goal: by end of Week 3 you can show Eyal something concrete.

Deliverables:

eval/week3_eval.md table: (V)



- `eval/week3_eval.md` table:

video	expected phase	predicted phase	notes
living-room	Calm	Calm	matches; low density
action clip	Dynamic	Dynamic	matches; high sf
crowded scene	Dense	Calm	threshold maybe too high

Also store per-video outputs in `outputs/`.

Week 4+ Roadmap (practical next steps, still "closed source only")

Week 4 — Upgrade LVLM output into structured fields (without heavy ML)

Right now you get a free-form paragraph. That's fine for a demo, but you eventually want stable fields.

Mission K — "LLM-as-parser" prompt (code + prompt design)

Add a second prompt that asks for JSON only:

Example (v1):

"Return JSON with: setting, main_entities, actions, emotion_words, interaction_level(0-3)…"

Deliverables:

- lvlm/prompts.py: PROMPT_SUMMARY_V1, PROMPT_STRUCTURED_V1 (V)
- lvlm/parse.py: robust JSON extraction (strip code fences, retry if invalid) (V)

This will let you:

- derive/validate interaction_density proxy (the agency of one who acts for another) if GCP signals are weak
- sanity-check your metadata

Week 4 — Add sanity checks + consistency guards

Mission L — Validator module

- if duration is 0 → fail
- if shot_count missing → mark as null and skip dependent features (or fallback)
- if any normalized feature is NaN → clamp/repair

Deliverable:

- algorithm/validate.py (V)
- Improved lvlm/client.py (V)
- Improved pipeline/test_lvlm.py (V)
- Improved pipeline/run_full_pipeline.py (V)
- Organized test scripts to a separate folder named: tests (V)
- Evaluation table for week 4

Fixed bugs:

- Duplicate frame extraction
- Handling the lvlm (testing) before integrating with GCP layer
- Handling LVLM inference with retries and fallbacks

This project could potentially be very helpful to blind and near-sighted people.(Visually impaired) We thought that maybe it will be beneficial for blind people, might add for a “Nice to have” feature. (Text to Speech) - first of all finish the project itself.

Todo: Optimization for GLM 4.6V (fixed prompts for the videos)
Todo: Build basic website
Todo: Write a Final Project book
Todo: prompt engineering
Todo: document failures , document unexpected output
Todo: make sure the website is basic, make sure I can run at least 50 videos for accuracy percentage. (document) (can up to 100) (document if there’s an improvement/downgrade)
Gaps Analysis: if it’s feasible
Todo: fallbacks (document whatever goes wrong or right), make sure the results are consistent
Todo: plan for end point presentation, plan if the presentation is enough.
If time is left: add Text to Speech feature
TOON - token oriented object notation (csv) - for the 50/100 video testing

הנה דוגמא לפורמט TOON ש-LLM עמור להסתדר אתו טוב יותר מאשר JSON.
בדוגמא: פרטים על אנשים - השורה הראשונה מגדירה את סוג הנתונים, כל שורה נוספת מייצגת אדם

```
users [2]{id,name,role}:
1,Alice,admin
2,Bob,user
```

Recommended flow in the updated full pipeline

1. read local video path
2. upload/check in GCS (Google cloud storage)
3. analyze with GCP (Google cloud platform)
4. compute VideoFeatures
5. validate VideoFeatures
6. run LVLM **summary inference** (סיכום הסקת מסקנות)
7. run LVLM **structured inference** (הסקת מסקנות מובנית)
8. parse structured response (ניתוח)
9. validate structured LVLM fields
10. run algorithm interpretation
11. validate normalized features
12. save VideoFeatures.json
13. save VideoInterpretation.json

🔥 The difference (very important)

Aspect	`gcp/features.py`	`algorithm/features.py`
Layer	Data extraction	Algorithm
Input	API annotations	JSON metadata
Output	VideoFeatures.json	Feature vector
Purpose	Build dataset	Prepare for model
Complexity	Heavy	Light

[GCP Layer]
VideoFeatures.json ← (src/gcp/features.py)

↓
[Algorithm Layer]
↓
Raw Features ← (src/algorithm/features.py)
↓
Normalized Features
↓
Score + Phase

Overall judgment of the LVLM test

This test proves:

- `prompts.py` works
- `client.py` works
- `parse.py` works
- `.env` loading works
- `GLM_API_KEY` is being picked up correctly
- frame extraction works
- retry logic works
- output saving works

That is a full LVLM module validation.

Updated flow:

video path
→ GCP upload/analyze
→ VideoFeatures.json
→ validate GCP features
→ extract LVLM frames once
→ LVLM summary
→ LVLM structured
→ parse + validate LVLM structured
→ algorithm raw features
→ validate raw features
→ normalize
→ validate normalized features
→ complexity + phase
→ VideoInterpretation.json

Week 5 — Improve thresholds and ranges using your own mini dataset

Still no “big dataset needed”, but you can tune safely.

Mission M — Numeric feature statistics

Original mission stays:

- run pipeline on ~20 clips (V)
- collect raw feature distributions (V)
- update normalization ranges in config.py (V)
- create evaluation/feature_stats.md (V)

Mission N — LVLM interaction-level tuning

New mission:

- collect expected vs predicted interaction level (V)
- identify mismatch cases (V)
- improve PROMPT_STRUCTURED_V1
- rerun on the same clips
- compare before/after

Deliverable:

evaluation/lvlm_interaction_eval.md
or add a section inside:
evaluation/week5_eval.md

Track A — Numeric feature tuning

GCP / algorithm features:

- shot_frequency
- object_entropy
- interaction_density
- human_presence_ratio

Track B — LVLM semantic tuning

Especially: lvlm_interaction_level

Week 6

- complexity_score becomes a continuous control knob

The system:

1. Takes a **video**
2. Extracts **visual metadata**
3. Uses **LVLM** to obtain semantic understanding
4. Computes **interpretable scene metrics**
5. Produces **structured JSON describing the scene**

Output = **metadata + interpretation**
The system analyzes short videos and produces an interpretable scene signature combining numeric metadata and LVLM semantic understanding.

- The project could potentially be very beneficial for blind people, as well as film makers, film students, directions, screenwriters, script writers, etc.

Clean implementation order for today

Step 1 - Create folders and empty files. (V)

Step 2 - Write config.py (V)

Include:

- normalization ranges
- weights
- thresholds
- frame settings
- model name

Step 3 - Write algorithm/features.py (V)

Compute:

- shot_frequency
- interaction_density

Step 4 - Write algorithm/normalize.py (V)

Step 5 - Write algorithm/complexity.py (V)

Step 6 - Write algorithm/phase.py (V)

Step 7

Create one example metadata JSON and run the algorithm only (V)

At this stage, don't even connect LVLM yet.
Just prove:

- raw features → normalized → score → phase

Determine config.py and compute algorithm scripts - Need to test that before moving on (V)

Readme:
VideoFeatures.json stores the metadata extracted from the video analysis tools and APIs, including duration, shot statistics, labels, and object-based scene statistics.

Interpretation file:

VideoInterpretation.json stores the Week 3 interpretation layer, including:

- the 4 core features (from VideoFeatures.json) (V)
- normalized feature values (normalize.py) (V)
- SceneComplexityScore (V)
- NarrativePhaseDetector (V)
- explanation fields (V)
- optional LVLM summary (GLM 4.6V API) (WIP)

`interaction_density`: a proxy for scene busyness based on tracked object segments per second

TODO:

* video id optimization: 09230-09250 (21 videos) - (V)
* fear of overfitting
* change config.py and retest (V)
* how it affects the token amount (optimization) - nothing changed, still 0.
* prepare questions if I have

Week 6 Goal

By the end of Week 6, you should have:

1. a clearer evaluation of LVLM `interaction_level`
2. improved LVLM prompt/structured output if needed
3. a basic website/demo interface that can present:
    - uploaded/analyzed video
    - `VideoFeatures.json`
    - `VideoInterpretation.json`
    - scene complexity score
    - narrative phase
    - LVLM summary
    - LVLM structured metadata

The website does not need to be perfect yet. It should be a presentable demo shell.

Track A — LVLM Interaction-Level Tuning

Mission A — Define the interaction-level scale clearly

Before testing more, lock down the meaning of the scale:
0 = no visible interaction
1 = weak / indirect interaction
2 = clear mutual interaction
3 = strong / intense interaction

Use this in: src/lvlm/prompts.py
Your prompt should force the LVLM to return:

```
{
 "interaction_level": 0,
 "interaction_evidence": "..."
}
```

The `interaction_evidence` field is important because it explains why the model chose that level.

Mission B — Create Week 6 LVLM interaction evaluation file

Create: evaluation/week6_lvlm_interaction_eval.md

Suggested structure:

Week 6 LVLM Interaction-Level Evaluation

Goal

Evaluate whether the LVLM `interaction_level` field matches human expectations after prompt tuning.

Interaction Scale

| level | meaning |

|------|---------|

| 0 | No visible interaction |

| 1 | Weak / indirect interaction |

| 2 | Clear mutual interaction |

| 3 | Strong / intense interaction |

Evaluation Table

| video_id | expected_interaction_level | predicted_interaction_level | match | interaction_evidence | notes |

|---------|----------------------------|-----------------------------|-------|----------------------|------|

| ACCEDE09230 | 0 | TODO | TODO | TODO | Quiet domestic scene, separate activities |

| ACCEDE09231 | 1 | TODO | TODO | TODO | Car scene, weak indirect interaction |

| ACCEDE09232 | 0 | TODO | TODO | TODO | One person walking, no direct interaction |

| ACCEDE09233 | 2 | TODO | TODO | TODO | Car scene with clearer interaction |

You can start with the same 4 videos, then expand to 8–10.

Mission C — Run LVLM tuning mode

Use the batch mode we designed - command:

python -m src.pipeline.run_batch_pipeline --videos_dir "C:\Users\Kim\OneDrive - Afeka College Of Engineering\Desktop\Final Project New\Final-Project\videos\videos" --limit 4 --mode lvlm_tuning

Later, use more videos - command:

python -m src.pipeline.run_batch_pipeline --videos_dir "C:\Users\Kim\OneDrive - Afeka College Of Engineering\Desktop\Final Project New\Final-Project\videos\videos" --limit 8 --mode lvlm_tuning

Do not run LVLM on all 21 immediately. It costs time and can hit rate limits.

Mission D — Compare expected vs predicted interaction level

For each video, inspect: outputs/<video_id>/VideoInterpretation.json

Check:
"lvlm_structured": {
"interaction_level": ...,
"interaction_evidence": "..."
}

Then fill the evaluation table manually.
Important cases from Week 4: ACCEDE09230 expected 0, ACCEDE09231 expected 1, ACCEDE09232 expected 0, ACCEDE09233 expected 2

These are your regression test cases.

Mission E — Tune prompt if mismatch remains

If the LVLM still predicts `0` too often, improve the structured prompt.
Prompt emphasis should be:

If people speak, look at each other, gesture toward each other, react to each other, or share an activity, do not return 0.

Also:

Driving together in a car may be level 1 or 2 depending on whether there is visible communication/reaction.

This should help with `ACCEDE09231` and `ACCEDE09233`.

Mission F — Decide final LVLM status

At the end of this track, write a short conclusion:

Conclusion

The LVLM structured output is useful for semantic metadata such as setting, entities, actions, emotion words, and scene summary. The `interaction_level` field improved after prompt tuning, but remains interpretive and should be treated as semantic metadata rather than a ground-truth measurement.

This is honest and academically safe.

Website Mission 6 — Documentation

Add to README:

Website Demo

The website displays analyzed video metadata generated by the pipeline.

Current demo fields:
- SceneComplexityScore
- NarrativePhaseDetector output
- GCP-based raw and normalized features
- LVLM free-text summary
- LVLM structured semantic metadata

Also mention:

The Demo Tool will run GCP/LVLM analysis **live** when uploading a video for analysis (a new video that has not been tested upon).

Until the end of the month - tuning the LVLM interaction lvl - retest on 21 videos.

TODO:

- Pay attention to which videos - he remembered, and which new videos - if he gives logical results.
- 50~ - tokens, costs, after testing. (30 additional videos which are new) - document.
- Its an R&D final project.
- Pay more attention on tuning.
- An research project for lab and optimization - a tool.
- Focus on tuning, retesting, and research.
- Check about publishing the project on afeka's website/forums.
- Schedule a meeting with Eyal and Dr. Tysh.
- Prompts will go to Final project book.

LVLM TODO:

1. Tune prompt now using the 4 known regression videos. (V)
2. Rerun the same 4 videos. (V)
3. Check whether the two mismatches improve. (V)
4. Only then expand to 8–10 videos.